Speaker Notes — Secure SDLC & Threat Modeling

NetworkOptix, November 2025 — speaker notes export, one section per slide

Slide 1. **Title Slide — Secure SDLC & Threat Modeling**

(No speaker notes on this slide)

Slide 2. **Agenda**

Hi everyone, and welcome! Today we'll look at how security fits naturally into the development process — not as a separate checklist at the end, but as something we can integrate from the very start.We'll begin with a quick overview of the Software Development Lifecycle (SDLC) and see where security belongs in each phase.Then we'll move into threat modeling — a structured way to think through potential risks before any code is written.To make this practical, we'll create a simple Data Flow Diagram (DFD) to map how data moves through a system. From there, we'll identify trust boundaries — the key transition points where data crosses between different trust levels — and see why they matter.After that, we'll apply the STRIDE framework to one boundary as a hands-on example.And finally, we'll wrap up with how to use the results — what each role can take away and how this process can evolve as your projects grow.The goal today isn't to memorize theory, but to understand a practical tool you can apply in your own projects to design more secure systems.

Slide 3. **What is SDLC**

Let's start simple — SDLC, or Software Development Life Cycle, is the process of planning, building, testing, deploying, and maintaining a system. In plain English — it's the story of your product from "idea" to "still running in prod somehow."Security in SDLC means we don't wait for things to break before fixing them. The earlier you catch an issue, the cheaper it is — that's the whole "shift-left" idea.Think of it as debugging your design.So, secure SDLC isn't about paperwork — it's about not spending Friday night chasing a breach alert.

Slide 4. **Security Across SDLC**

SDLC is made of stages — and security belongs in each of them:Requirements: define what "secure" actually means for your system.Design: predict what could go wrong — this is where Threat Modeling lives.Implementation: write code that doesn't introduce new vulnerabilities.Testing: perform DAST, fuzzing, and negative testing to catch logic gaps.Operations: monitor, harden, and respond before attackers do.In short: plan it, build it, break it (safely), and protect it.Security isn't a checkbox at the end — it's a habit across the whole lifecycle.

Slide 5. **Threat Modeling Basics**

According to OWASP, threat modeling is a structured method for identifying, prioritizing, and mitigating potential threats to a system — and for evaluating how effective our defenses actually are.Threat modeling sounds complicated, but it’s really structured common sense. You take your system, map how data moves, and ask: “What happens if someone messes with this?”Today we’ll do exactly that:Build a Data Flow Diagram,Mark Trust Boundaries,And apply STRIDE to identify threats.It’s like a code review, but for your design.

Slide 6. **Data Flow Diagram**

Since AI is everywhere lets imagine we want to implement a new llm based application with our own hosted LLM. So here’s what a Data Flow Diagram, or DFD, looks like in the context of an LLM-based application.Think of it as a high-level map of how information moves through the system — who talks to whom, and what crosses those lines.In this example, we have an LLM (Large Language Model) at the center — the brain of our app. On the left, we have External Prompt Sources — that’s where all the inputs come from: real users, web interfaces, APIs, even emails or chatbots. They send prompts, and the LLM returns responses.On the right model interacts with internal systems:Server-side functions, which the LLM can call to perform actions — think of things like “get user data,” “send a message,” or “run an analysis.”Private data sources, like internal databases, document repositories, or APIs, which the LLM can query or update.The arrows show data flows — for example, prompts and responses between users and the LLM, requests and outputs between the LLM and backend functions, or read/write operations with private data.So in threat modeling, this kind of diagram helps us visually identify where trust changes, where inputs need validation, and where an attacker might sneak in.In short: a DFD is not just boxes and arrows — it’s your system’s “X-ray” view. It shows how data moves and where it might misbehave if nobody’s watching.

Slide 7. **Trust Boundaries**

Now that we've mapped out the main components, the next step is to add trust boundaries to our model.A trust boundary marks the point where the level of trust changes — for example, between an unauthenticated and an authenticated user, or between an end user and an internal system. Whenever data crosses one of these boundaries, it must be validated, authenticated, or access-controlled, otherwise attackers could inject malicious input or corrupt data.So we should declare following trust boundaries:TB01 (External Prompt Sources → LLM): This is the point where data enters the system from completely external sources: human users, emails, web input, even other automated systems. None of that input is inherently trustworthy. Why it's a trust boundary: because before TB01, the content lives in environments we do not control; after TB01, it becomes something our system will start processing. So this is where we must assume: prompts may be malicious, manipulated, or crafted to influence the model.Example attacks:Prompt Injection: a user hides malicious instructions inside a prompt to make the LLM reveal secrets or override system rules.Malicious Output: the LLM generates unsafe text — such as a JavaScript snippet that causes Cross-Site Scripting (XSS) when displayed in a web UI.TB02 (LLM → Server-side Functions): Here the LLM is no longer just "chatting," it's asking our backend logic to do real work through server-side functions. Why it's a trust boundary: the moment model output can cause server actions, we're switching from "text generation" to "performing operations in our infrastructure." That requires strict policy: the LLM cannot be treated as an inherently trusted decision-maker. We have to check: Is this function allowed? For this user? With these parameters?Important nuance: we are not saying the LLM is evil. We're saying: the LLM can be influenced by untrusted input (from TB01), so we cannot automatically trust what it asks us to execute.Example attacks:Command Injection: the model's response includes shell commands that get executed by the server (e.g., rm -rf /).API Misuse: the LLM constructs unintended or unauthorized API calls, such as changing user roles or deleting data.Function Call Abuse: the model triggers functions like exec(), eval(), or automation workflows with unvalidated parameters.TB03 (LLM → Private Data Sources): Here the LLM is requesting access to private/internal data repositories ("Private Data Sources" in the diagram). That may include confidential or sensitive data. Why it's a trust boundary: before TB03, data is stored under controlled policies; after TB03, that data could be exposed back through the model. So we must gate access, control what the model is allowed to read/write, and make sure we don't accidentally leak sensitive internal data back to an external prompt source.Example attacks:Data Exfiltration: a prompt or chain of interactions causes the LLM to leak confidential data retrieved from a database.Unauthorized Queries: an attacker convinces the LLM to read or modify data they shouldn't have access to.Training Data Leakage: the model reveals snippets of proprietary or personally identifiable information (PII) memorized during training.

Slide 8. **STRIDE**

Now that we've defined our data flow diagram and marked all the trust boundaries, the next logical step is to identify potential threats that exist along those boundaries.To do that, we can use a structured methodology called STRIDE. It helps us think systematically about how our system could be attacked, and what types of weaknesses we should be aware of.STRIDE stands for six categories of security threats, each linked to a specific property that it violates:S — Spoofing (Authentication): Pretending to be someone or something else — for example, logging in with another user's credentials or forging an API identity.T — Tampering (Integrity): Modifying or manipulating data — such as altering database entries, intercepting and changing data in transit, or injecting malicious content into an LLM prompt.R — Repudiation (Non-repudiation): Performing an action and later denying it — for example, submitting harmful input and then claiming "it wasn't me." This highlights the need for strong logging and traceability.I — Information Disclosure (Confidentiality): Gaining access to information that should remain private — like an LLM unintentionally revealing sensitive data or training content.D — Denial of Service (Availability): Overloading a system or model so it can't respond to legitimate requests — such as flooding an LLM with prompts that exhaust memory or compute resources.E — Elevation of Privilege (Authorization): Gaining higher-level permissions — for instance, manipulating an LLM or API to execute actions reserved for admins or backend services. How it's appliedWhen performing threat modeling, we go through each component and trust boundary in our DFD, and systematically ask:"Could Spoofing happen here? Could someone Tamper with data? Could there be a risk of Information Disclosure?"By doing this for each STRIDE category, we can map out which threats apply to each boundary (TB01, TB02, TB03), For now instead of repeating STRIDE for every boundary, we'll focus on just one — that's enough to show the logic. The point is to demonstrate how STRIDE helps structure your thinking. But in real projects, you'd repeat this analysis for each boundary

Slide 9. **STRIDE — TB1: External Endpoints**

Now that we know how STRIDE applies to our model, let's go deeper into Trust Boundary 1 (TB01) — the point between external users and the LLM. Let's go through STRIDE step by step, keeping it simple by applying strength and weaknesses. This helps us understand where security controls exist and where vulnerabilities might appear.Spoofing (pretending to be someone else): Our biggest risk here is prompt injection — a malicious user crafting text that tricks the model or overrides the system prompt. Think of it like social engineering for AI — it's still text, but it's trying to hijack the context.Tampering (messing with integrity): Attackers might not change the code directly but could modify the parameters that control model behavior — temperature, length, even which model variant is called. It's subtle, but those tweaks can make the system behave unpredictably.Repudiation (denying actions): Ideally, we have authentication and logging, so every prompt is linked to a user. But if those aren't enforced properly, users could do something shady, then say "wasn't me."Information Disclosure (leaking data): This is a big one — users can accidentally paste secrets or internal text into the chat. It's not the LLM's fault, but it's still a data-leak path. In short: sometimes the user is the vulnerability.Denial of Service: Here we assume rate limits or authentication keep things stable. But if those controls fail, a bot sending 1000 prompts per second can easily overwhelm the API.Elevation of Privilege: Again, the controls here are mostly external — identity, roles, access tokens. So we mark it as a "strength assumed," but it really depends on how solid our surrounding infrastructure is.In short: TB01 is the front door — everything untrusted enters here. Most problems aren't exotic hacks; they're just people and inputs we can't trust.

Slide 10. **STRIDE — TB1: LLM**

The LLM is where most people expect “magic,” but from a security perspective, it’s more like controlled chaos. Let’s unpack each STRIDE category again, but this time, see why many of them are greyed out:Spoofing: The model doesn’t manage user identities or sessions. It just takes text and predicts more text. So spoofing here doesn’t really apply — it can imitate someone’s writing style, but that’s not authentication.Tampering: The model doesn’t store state you can corrupt directly. Still, poisoned training data or manipulated prompts could alter how it behaves — but that’s an upstream supply-chain risk, not something STRIDE can fix here.Repudiation: No logs, no audit trail. The LLM doesn’t know who sent what — which means accountability has to live outside, at the service or API level.Information Disclosure: Here’s the hot zone — VULN04: models may leak sensitive data. They might recall snippets from training, reproduce confidential context, or echo something a user pasted earlier. This is where research is still catching up.Denial of Service: Technically possible — prompt loops or massive inputs can eat resources — but again, the mitigation is external: request limits, quotas, and timeouts.Elevation of Privilege: The model can’t “gain” new privileges, but its outputs can influence privileged systems if those systems blindly trust its responses. So it’s not direct privilege escalation — it’s indirect, through automation.So overall, when we apply STRIDE to the LLM, we change how we think: It’s less about “can it authenticate or block access” and more about “what could it accidentally reveal or cause downstream.” That’s why the table is mostly grey — not because it’s safe, but because the real controls are around it, not inside it.

Slide 11. **From Threats to Action**

We’ve just seen how STRIDE helps us find weaknesses across boundaries — now let’s talk about what happens next and why this actually matters.Threat modeling isn’t only about drawing DFDs. Its real power is how different roles use it in their daily work.🤷 Developers: For devs, threat modeling is like a lens that makes risks visible before the first commit. It helps you think like an attacker — how inputs can be misused, how APIs might be abused. The next step? Take the identified threats and bake mitigations right into the design: sanitize inputs, enforce auth early, avoid insecure defaults. The advantage: you prevent whole classes of vulnerabilities before they ever exist.🧪 QA Engineers: For QA, it’s a roadmap of what to test negatively. Each threat we found can become a scenario: invalid tokens, oversized payloads, weird edge cases. It deepens test coverage and brings QA closer to security testing. The next step is to extend your existing tests with these edge cases — to see how the system fails, not just how it works.🏗️ Architects: For architects, threat modeling validates design resilience. It shows where trust boundaries leak or where external services might expose sensitive data. The next step is to refine architecture: add isolation, adjust data flows, enforce encryption. The benefit is foresight — you strengthen the design before implementation costs skyrocket.📊 PMs / Product Owners: For PMs, the outcome is prioritization with context. Now you can rank risks by business impact — which ones need fixing now, which can wait. The next step? Turn the threat list into a small security backlog, align it with sprint planning. The benefit: transparent trade-offs between security, delivery speed, and product goals.And yes — usually AppSec owns the process. Ideally, threat modeling runs regularly — at least for every new feature or big architectural change. The goal isn’t to slow down delivery but to help teams find the right balance between security depth and development speed.So the answer to “what do we do with it?” is: turn insights into routine decisions. Threat modeling is not a one-time event — it’s how we bring security awareness into everyday design, testing, and planning.

Slide 12. **Other Frameworks**

Up to this point, we've been working with STRIDE — and that's perfect for learning and for fast, focused sessions. But in real life, teams evolve. Once threat modeling becomes part of your regular SDLC, you start asking "how do we scale this?" or "how do we link this with enterprise risk management?"That's where other frameworks come in.PASTA (Process for Attack Simulation and Threat Analysis): This one's more formal and risk-driven — it walks you through seven stages, starting from understanding business objectives all the way to defining mitigation strategies. It's heavier than STRIDE but fits well for companies that need measurable, auditable reporting or want to align security with business goals.OCTAVE (Operationally Critical Threat, Asset, and Vulnerability Evaluation): This shifts the focus from individual applications to the whole organization — it looks at assets, processes, and operational risks. It's great for large environments where you need to prioritize across multiple systems or departments.VAST (Visual, Agile, and Simple Threat): This one's all about scalability. It allows different teams or product areas to run their own mini-threat-models and roll them up into one big picture. Perfect for microservices or distributed development setups.In practice, many modern teams do a hybrid approach:start with STRIDE for clarity and speed;adopt PASTA or OCTAVE elements once governance and risk reporting become important;use VAST when you want to make threat modeling part of every team's agile workflow.The point is — there's no single "right" framework. STRIDE teaches you how to think; these others help you scale that thinking across products and organizations.

Slide 13. **Key Takeaways**

First — security starts at design. Threat modeling is about prevention. Catching risks during design saves massive effort later, when those same issues could turn into production incidents.Second — do it together, and involve the security team early. The goal of the security team isn't to review your work after the fact or to slow you down. The goal of security team is to amplify what you're already doing — to help you see patterns that others have faced before, share reusable controls, and give feedback that makes your solution more resilient.The way of thinking that security team is a partner that helps translate "is this safe?" into clear, practical next steps.Third — make it continuous. Each new feature, integration, or third-party dependency can shift the threat landscape. Use quick, lightweight modeling sessions as a regular check-in — part of the same rhythm as design reviews or sprint planning.In short: Threat modeling isn't about bureaucracy — it's about collaboration. We build better, safer systems when everyone, including the security team, is part of the design conversation from the start.

Slide 14. **Thank you!**

(No speaker notes on this slide)